

Midterm Exam 1

(11:05 AM – 12:20 PM : 75 Minutes)

- This exam will account for either 15% or 30% of your overall grade depending on your relative performance in midterm exam 1 and midterm exam 2. The higher of the two scores will be worth 30% of your grade, and the lower one 15%.
- There are four (4) questions worth 75 points in total. Please answer all of them in the spaces provided.
- There are twenty-two (22) pages, including nine (9) blank pages and one (1) page of appendices. Please use the blank pages if you need additional space for your answers.
- The exam is **open slides** and **open notes** (including **scribe notes**). But **no books** and **no computers** are allowed.

GOOD LUCK!

Question	Page(s)	Part(s)	Points	Score
1. Alignment	2 – 5	(a) – (b)	$5 + 20 = 25$	
2. Does Master Theorem Apply?	7 – 11	(a) – (c)	$6 + 8 + 6 = 20$	
3. Strassen with Memory Leaks	13 – 18	(a) – (c)	$4 + 10 + 6 = 20$	
4. Karatsuba's Algorithm	20	–	10	
Total			75	

NAME: _____

SBU ID: _____

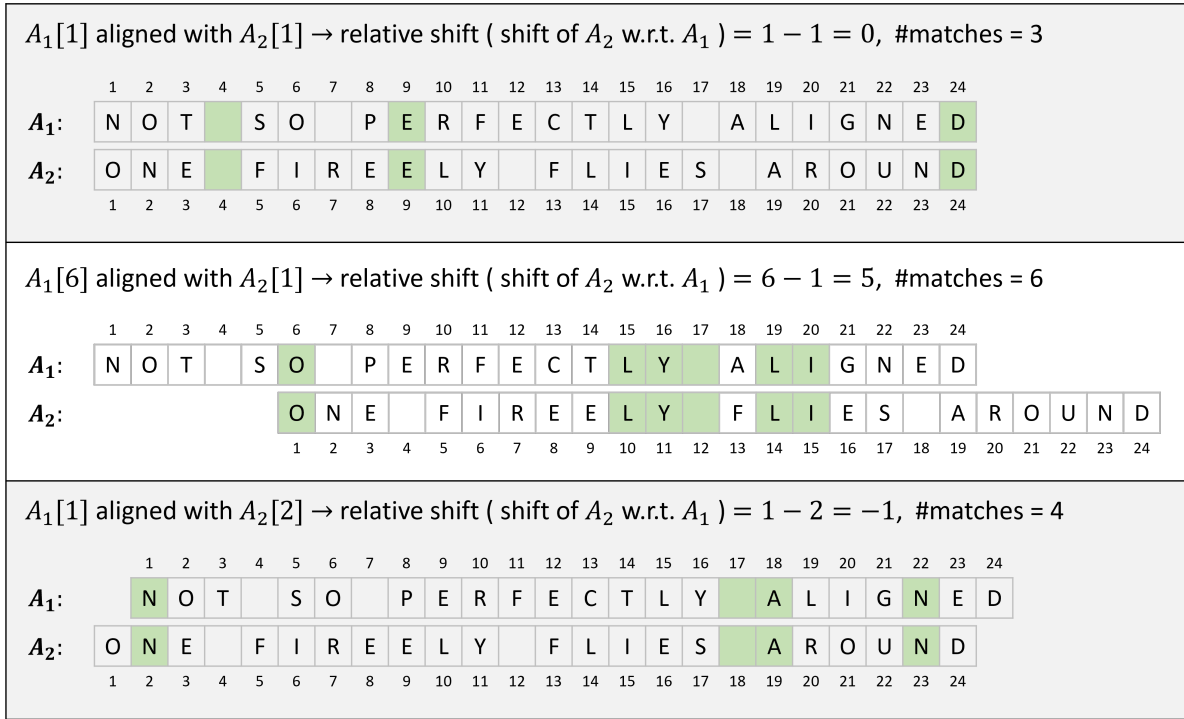


Figure 1: [Question 1] Three different alignments of two arrays $A_1[1 : 24]$ and $A_2[1 : 24]$ containing English text. Aligned cells with matching symbols are marked with green.

QUESTION 1. [25 Points] Alignment. You are given two arrays $A_1[1 : n]$ and $A_2[1 : n]$ of the same length $n \geq 1$. Each cell of the arrays contains either a space symbol (i.e., blank) or one of the 26 uppercase letters of the English alphabet.

This question asks you to align A_1 and A_2 such that the number of aligned cell pairs with matching symbols is maximized. Figure 1 shows three example alignments of two text arrays A_1 and A_2 of the same length 24. The top part of the figure shows that aligning A_1 and A_2 at index 1 produces 3 matches (shown in green). The middle part shows that aligning index 6 of A_1 with index 1 of A_2 results in 6 matches. Finally, the bottom part shows that when index 1 of A_1 is aligned with index 2 of A_2 , there are 4 matches.

When $A_1[i]$ is aligned with $A_2[j]$, we say that the alignment is obtained when A_2 is right-shifted by $i - j$ units relative to A_1 . A negative value of $i - j$ indicates a left shift.

This question asks you to find a relative shift that results in the maximum number of symbol matches between A_1 and A_2 . Note that multiple relative shifts may produce the same number of maximum matches. It suffices to find any one of them.

Now answer the following questions.

- (a) [**5 Points**] Give a $\Theta(n^2)$ -time algorithm to find a relative shift that produces the maximum number of symbol matches between $A_1[1 : n]$ and $A_2[1 : n]$.

page intentionally left blank (use for your answers, if needed)

- (b) [**20 Points**] Solve the same problem as in part (a) but make sure that your algorithm runs in $\Theta(n \log n)$ time.

page intentionally left blank (use for your answers, if needed)

QUESTION 2. [20 Points] Does Master Theorem Apply? Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a positive function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ a T\left(\frac{n}{b}\right) - f(n), & \text{otherwise.} \end{cases}$$

Clearly, this recurrence is not in the form solvable by the Master Theorem (e.g., see the Appendix) because of the $-f(n)$ in the recursive part of it.

This question asks you to transform this recurrence into a form solvable using the Master Theorem when $f(n) = n^p$ for some real-valued p .

- (a) [6 Points] In the recurrence for $T(n)$ above make the substitution $S(n) = T(n) - cf(n)$, where c is a constant. For what value of c do you get a recurrence of the following form for $S(n)$?

$$S(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ a S\left(\frac{n}{b}\right), & \text{otherwise.} \end{cases}$$

What relationship between p and $\log_b a$ must hold for constant c to exist?

page intentionally left blank (use for your answers, if needed)

- (b) [**8 Points**] Solve the recurrence for $S(n)$ from part (a) using the Master Theorem, and use the solution for $S(n)$ to show that $T(n) = \Theta \left(n^{\log_b a} - \left(\frac{b^p}{b^p - a} \right) n^p \right)$.

page intentionally left blank (use for your answers, if needed)

(c) [**6 Points**] Use your results from parts (a) and (b) to solve the following recurrences.

$$T_1(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ 3 T_1\left(\frac{n}{2}\right) - n, & \text{otherwise.} \end{cases}$$

$$T_2(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ 4 T_2\left(\frac{n}{2}\right) - n^2, & \text{otherwise.} \end{cases}$$

$$T_3(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ 5 T_3\left(\frac{n}{3}\right) - n^2, & \text{otherwise.} \end{cases}$$

page intentionally left blank (use for your answers, if needed)

<pre> MM-STANDARD(X, Y, Z) Input: $n \times n$ matrices X and Y, where n is a power of 2 Output: $Z \leftarrow XY$ 1. if X is a 1×1 matrix then $Z \leftarrow Z + XY$ 2. else { Recursion: 8 recursive calls } 3. MM-STANDARD(X_{11}, Y_{11}, Z_{11}) MM-STANDARD(X_{12}, Y_{21}, Z_{11}) MM-STANDARD(X_{11}, Y_{12}, Z_{12}) MM-STANDARD(X_{12}, Y_{22}, Z_{12}) MM-STANDARD(X_{21}, Y_{11}, Z_{21}) MM-STANDARD(X_{22}, Y_{21}, Z_{21}) MM-STANDARD(X_{21}, Y_{12}, Z_{22}) MM-STANDARD(X_{22}, Y_{22}, Z_{22}) </pre>	<pre> MM-STRASSEN(X, Y, Z) Input: $n \times n$ matrices X and Y, where n is a power of 2 Output: $Z \leftarrow XY$ 1. if X is a 1×1 matrix then $Z \leftarrow XY$ 2. else 3. Allocate twenty-one $\frac{n}{2} \times \frac{n}{2}$ matrices $X_0, \dots, X_6, Y_0, \dots, Y_6, P_0, \dots, P_6$ { Compute input matrices } 4. Compute $X_0, \dots, X_6$ from the quadrants of X, and $Y_0, \dots, Y_6$ from the quadrants of Y { Recursion: 7 recursive calls } 5. MM-STRASSEN(X_0, Y_0, P_0) MM-STRASSEN(X_1, Y_1, P_1) MM-STRASSEN(X_2, Y_2, P_2) MM-STRASSEN(X_3, Y_3, P_3) MM-STRASSEN(X_4, Y_4, P_4) MM-STRASSEN(X_5, Y_5, P_5) MM-STRASSEN(X_6, Y_6, P_6) { Compute output matrix } 6. Compute the quadrants of Z from $P_0, \dots, P_6$ </pre>
--	--

Figure 2: [Question 3] MM-STANDARD is the standard recursive matrix multiplication algorithm that does not allocate any extra space, and MM-STRASSEN is a variant of Strassen’s algorithm that allocates 21 matrices in each recursive call (see Line 3). Note that our MM-STRASSEN code shown here mistakenly does not deallocate those matrices.

QUESTION 3. [20 Points] Strassen with Memory Leaks. Figure 2 shows MM-STRASSEN which is a variant of Strassen’s algorithm that allocates 21 matrices in each recursive call (see Line 3). Our code, however, has memory leak as it mistakenly never deallocates the memory it allocates. As a result, all memory space it allocates during its execution may remain allocated even after it terminates.

In order to reduce the memory problem of MM-STRASSEN we have enlisted the help of MM-STANDARD (shown in Figure 2) which is the standard recursive matrix multiplication algorithm that does not allocate any extra space at all. The result is a hybrid algorithm called MM-STRASSEN-HYBRID (see Figure 3). MM-STRASSEN-HYBRID calls MM-STANDARD-HYBRID to solve its 7 subproblems recursively and MM-STANDARD-HYBRID, in turn, calls MM-STRASSEN-HYBRID to solve its 8 subproblems recursively. While MM-STRASSEN-HYBRID allocates extra space, MM-STANDARD-HYBRID does not. So, this new hybrid algorithm allocates extra space only at every other level of recursion. The hope is that the hybrid algorithm will allocate less memory.

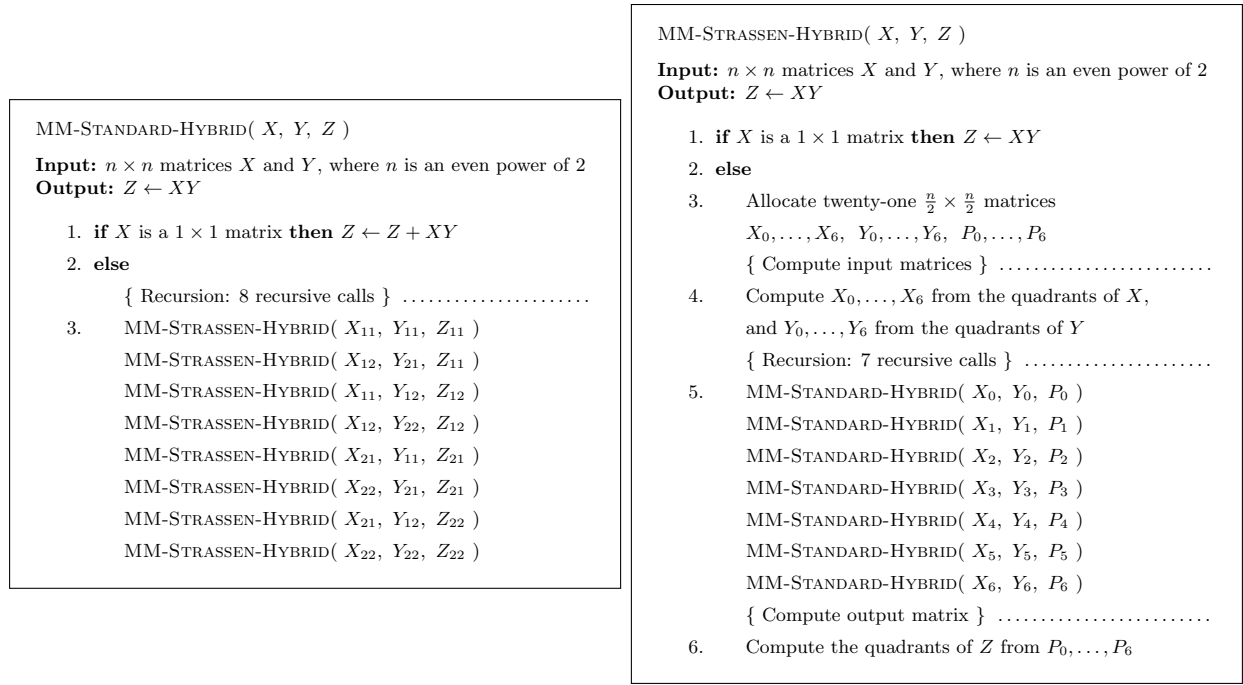


Figure 3: [Question 3] In an effort to reduce the space usage of our buggy MM-STRASSEN code from Figure 2, we have created MM-STRASSEN-HYBRID which calls MM-STANDARD-HYBRID to solve its 7 subproblems recursively and MM-STANDARD-HYBRID, in turn, calls MM-STRASSEN-HYBRID to solve its 8 subproblems recursively.

(a) [4 Points] Write a recurrence relation to compute the total space $S(n)$ allocated by MM-STRASSEN (see Figure 2) when multiplying two $n \times n$ matrices. Solve the recurrence.

page intentionally left blank (use for your answers, if needed)

- (b) [**10 Points**] Write a recurrence relation to compute the total space $S_h(n)$ allocated by MM-STRASSEN-HYBRID (see Figure 3) when multiplying two $n \times n$ matrices. Solve the recurrence.

page intentionally left blank (use for your answers, if needed)

- (c) [**6 Points**] Your solutions from parts (a) and (b) should imply that MM-STRASSEN-HYBRID uses asymptotically more space than MM-STRASSEN. Explain why this is not a discrepancy.

page intentionally left blank (use for your answers, if needed)

QUESTION 4. [10 Points] Karatsuba's Algorithm. Recall that when multiplying two n -bit integers x and y , where n is a power of 2, Karatsuba's algorithm splits both x and y in the middle and thus creating four $\frac{n}{2}$ -bit integers x_L , x_R , y_L , and y_R as shown below.

$$\begin{array}{lcl} x = & \overbrace{\boxed{x_L} \boxed{x_R}}^{\substack{\frac{n}{2} \text{ bits} & \frac{n}{2} \text{ bits}}} & = 2^{n/2} x_L + x_R \\ y = & \underbrace{\boxed{y_L} \boxed{y_R}}_{n \text{ bits}} & = 2^{n/2} y_L + y_R \end{array}$$

Then Karatsuba observes that

$$xy = \left(2^{\frac{n}{2}} x_L + x_R\right) \left(2^{\frac{n}{2}} y_L + y_R\right) = 2^n x_L y_L + 2^{\frac{n}{2}} ((x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R) + x_R y_R$$

So, Karatsuba's algorithm recursively solves the following three subproblems:

- $x_L y_L$: the product of two $\frac{n}{2}$ -bit integers
- $x_R y_R$: the product of two $\frac{n}{2}$ -bit integers
- $(x_L + x_R)(y_L + y_R)$: the product of two $(\frac{n}{2} + 1)$ -bit integers, as the sum of two $\frac{n}{2}$ -bit integers (i.e., $x_L + x_R$ and $y_L + y_R$) can be up to $(\frac{n}{2} + 1)$ -bits long

So, the recurrence relation describing the running time $T(n)$ of Karatsuba's algorithm should be as follows.

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1, \\ 2 T\left(\frac{n}{2}\right) + T\left(\frac{n}{2} + 1\right) + \Theta(n), & \text{otherwise.} \end{cases}$$

Show that the product of two $(\frac{n}{2} + 1)$ -bit integers can be reduced to the product of two $\frac{n}{2}$ -bit integers plus $\Theta(n)$ additional work. In other words, show that $T\left(\frac{n}{2} + 1\right) = T\left(\frac{n}{2}\right) + \Theta(n)$, and so the recurrence relation for $T(n)$ above can be rewritten as

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1, \\ 3 T\left(\frac{n}{2}\right) + \Theta(n), & \text{otherwise.} \end{cases}$$

page intentionally left blank (use for your answers, if needed)

APPENDIX: RECURRENCES

Master Theorem. Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be an asymptotically nonnegative function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise,} \end{cases}$$

where, $\frac{n}{b}$ is interpreted to mean either $\lfloor \frac{n}{b} \rfloor$ or $\lceil \frac{n}{b} \rceil$. Then $T(n)$ has the following bounds:

Case 1: If $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.

Case 2: If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some constant $k \geq 0$, then $T(n) = \Theta(f(n) \log n)$.

Case 3: If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$.

APPENDIX: COMPUTING PRODUCTS

Integer Multiplication. Karatsuba's algorithm can multiply two n -bit integers in $\Theta(n^{\log_2 3}) = \mathcal{O}(n^{1.6})$ time (improving over the standard $\Theta(n^2)$ time algorithm).

Matrix Multiplication. Strassen's algorithm can multiply two 2×2 matrices using 7 multiplications, and two $n \times n$ matrices in $\Theta(n^{\log_2 7}) = \mathcal{O}(n^{2.81})$ time (improving over the standard $\Theta(n^3)$ time algorithm).

Polynomial Multiplication. One can multiply two n -degree polynomials in $\Theta(n \log n)$ time using the FFT (Fast Fourier Transform) algorithm (improving over the standard $\Theta(n^2)$ time algorithm).